



Unidad Didáctica 6: DATOS BALANCEADOS

Calibración, métricas y explicabilidad de los
modelos

ÍNDICE

Contenido

CONTENIDO	2
1. INTRODUCCIÓN.....	1
2. OBJETIVOS	2
2.1. OBJETIVO GENERAL	2
2.2. OBJETIVOS ESPECÍFICOS.....	2
3. INTRODUCCIÓN: BALANCEO DE DATOS	3
4. SUBMUESTREO DE DATOS.....	4
5. NEARMISS.....	5
6. SOBREMUESTREO DE DATOS	6
6.1. EJEMPLO DE DATOS	6
6.2. MODELO BÁSICO	9
6.3. MODELO NEARMISS	13
7. SOBRE MUESTREO.....	17
8. NOTEBOOK	19

1. Introducción

¡Bienvenido a la asignatura de calibración, métricas y explicabilidad de datos!

Este documento de contenido teórico complementa y compila la información relacionada con la unidad didáctica 6 de la asignatura de "Calibración, métricas y explicabilidad de los modelos" del segundo cuatrimestre del Máster Universitario en Inteligencia Artificial (MUIA).

La inteligencia artificial (IA) es un campo de la informática que busca desarrollar sistemas capaces de realizar tareas que normalmente requieren de la inteligencia humana. Utiliza algoritmos y modelos matemáticos para enseñar a las máquinas a aprender, razonar y tomar decisiones. Uno de los enfoques más destacados en la IA es el aprendizaje automático, donde los sistemas pueden mejorar su rendimiento a través de la experiencia y la retroalimentación.



Ten en cuenta

El módulo 5 de esta asignatura es la primera parte de un caso de uso completo que abarca 4 sesiones de 90 minutos. Este documento complementa las sesiones dadas por el profesor Dr. Alberto Partida. El diseño de esta asignatura incluye elementos participativos y de trabajo en grupo, muy similares a la realidad que el estudiante encontrará en el mundo profesional. Este documento contiene explicaciones sobre el contenido de la asignatura y las tareas a realizar por los estudiantes. Se recomienda trabajar con los módulos 5 y 6 conjuntamente.

2. Objetivos

2.1. Objetivo general

El objetivo general de este módulo es enfrentarse a un caso donde el conjunto de datos es balanceado. Podemos entender como un conjunto de datos altamente balanceado como aquel dataset que vayamos a usar para una clasificación donde predomine un valor de la clasificación sobre otros.

En la mayoría de los casos de uso reales, se intenta predecir o detectar algún valor poco frecuente en los datos.

En la actualidad casos como la detección de fraude, la detección de enfermedades y muchos otros, La mayoría de los eventos siempre tienen el mismo valor, por lo que no sirven para poder entrenar el modelo. Un porcentaje menor al cuatro, 5% pueden servir para detectar este tipo de patrones.

2.2. Objetivos específicos

Los objetivos específicos de este módulo de caso de uso completo son los siguientes:

- Entender lo que es el balanceo de datos y su Retos.
- Utilizar técnicas de submuestreo de datos en ejemplos prácticos.
- Utilizar técnicas de sobre muestreo de datos en ejemplos prácticos.
- Comparar ambas técnicas en ejemplos prácticos.

3. Introducción: Balanceo de datos

En la mayoría de los casos, dentro de los retos que nos encontramos como científicos de datos, consiste en el desbalanceo. De unos datos los cuales, desde el punto de partida, no están bien balanceados. Normalmente en el Mundo real la mayoría de los eventos siempre tienen el mismo valor, y solo un pequeño porcentaje de ellos son los eventos que queremos detectar y analizar dentro de ese conjunto de datos. Desde que estamos ante ese balanceo incorrecto de los datos, tenemos una muestra muy pequeña de los datos que queremos encontrar dentro de todos los eventos del sistema.

Para que tengáis un contexto en entornos como la ciberseguridad, se intenta detectar patrones de ataque en unas tasas de 1 a 10.000. En la detección de fraude se basa en tasas de detección de 1 a 10 o 100. Y en entornos de Medicina esas tasas pueden ser del mismo calibre. Este es un reto muy importante porque si estos datos no se balancean correctamente, los modelos algorítmicos se convertirán en vagos y, por tanto, siempre intentarán predecir que no han detectado este tipo de anomalía. El reto consiste en balancear correctamente las muestras para forzar o animar a los modelos algorítmicos a detectar este tipo de anomalías dentro del conjunto tan elevado de datos. Un ejemplo sería si utilizamos una técnica normal para detección de fraudes en tarjetas de crédito. El modelo siempre dirá que no hay ningún tipo de fraude. ¿Por qué? Porque solo un 0,1% de los movimientos de tarjeta tendrán fraude y, por tanto, desde el punto de vista matemático, es más fácil para el modelo decir que no hay ni un solo fraude.



Recomendación práctica 1

Se recomienda que el estudiante mantenga un cuaderno de bitácora. Este diario de su progreso no será evaluado inicialmente, pero supone una gran herramienta para que la estudiante pueda monitorizar su progreso, en términos de esfuerzo y tiempo invertido, en el análisis de este caso de uso y en su experiencia dentro de un equipo de proyecto. Aquellos estudiantes para los que este módulo es su primer contacto con la ciberseguridad, la recomendación incluye el control del tiempo que dedican individualmente para leer sobre principios de ciberseguridad y su actual relevancia en el mundo digitalizado.

4. Submuestreo de datos

El primer método para enfrentarnos a este reto es reducir el conjunto de datos de entrenamiento de la muestra, que tiene más datos, reducir el número de ceros de la muestra, lo que nos permite ponderar. Y hacer más visibles las muestras que tienen ese valor deseado para poder compararlos con ellos.

Con esta técnica lo que pretendemos es reducir el número de datos que sirven para entrenar el modelo. Esta reducción no será equitativa entre las muestras a detectar y el resto de las muestras. Lo que vamos a hacer es reducir el número de muestras que no tienen un valor para nuestro modelo, es decir, aquellas muestras que dan un valor cero. Nuestro objetivo es maximizar la información de las muestras a las que predecir, que son las de valor 1.

Esta técnica lo que nos permite es de una forma muy simple, aumentar la información que tiene el modelo sobre las variables a las que queremos detectar. Esto conlleva que perdemos algún tipo de predicción de las muestras anteriores, ya que reducimos el número de muestras en el entrenamiento.

La forma más fácil, consiste en dividir la muestra en dos grupos donde los datos se dividen en función del valor de la clase mayoritaria. Una vez realizada esta división, crea un dataset de datos donde el número de filas recogidas de cada uno de estos dos grupos sea exactamente igual.

Podemos pasar de una muestra altamente balanceada con 400 muestras con el valor 0 y 10000 de valor 1 a un dataset con 400 muestras con valor 0 y 400 muestras con valor 1. De esa forma, ya disponemos de una muestra correctamente balanceada. Sin embargo, esta técnica aleatoria no es la más adecuada porque no permite extraer las muestras más adecuadas.

Para ello, vamos a usar una técnica llamada Nearmiss para seleccionar adecuadamente las muestras.

5. NearMiss

Como ejemplo, utilizaremos una técnica conocida como NearMiss para la paridad entre ambos valores de la variable target.

NearMiss es una familia de métodos de submuestreo que seleccionan ejemplos de la clase mayoritaria en función de su distancia (euclidiana) con respecto a los ejemplos de la clase minoritaria. La idea básica es conservar los ejemplos que están cerca de la clase minoritaria y descartar los ejemplos que están lejos de ella.

Para más información sobre esta técnica, podéis consultar aquí:

https://imbalanced-learn.org/stable/auto_examples/under-sampling/plot_illustration_nearmiss.html#sphx-glr-auto-examples-under-sampling-plot-illustration-nearmiss-py

6. Sobremuestreo de datos

La otra técnica utilizada es generar más datos sintéticos de los datos, que son pequeños en el muestreo, es decir, aumentaremos el número de datos con valor 1 para que la muestra se vuelva balanceada. Para esta técnica, igual que en el caso anterior, existen muchas técnicas matemáticas para conseguir que se generen datos. Similares a los anteriores. 1 de ellos es generar muestras muy similares de forma matemática a las muestras de asistentes. Esta técnica es más peligrosa que la anterior, ya que generamos datos que no son reales y, por tanto, probablemente sean del valor de fraude. Pero no sé, es asegurado desde el punto de vista real.

6.1. Ejemplo de datos

Para la realización de la práctica utilizaremos un dataset de detección de fraude en tarjetas de crédito que se encuentra en kaggle.

<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud/data>

MACHINE LEARNING GROUP - ULB AND 1
COLLABORATOR · UPDATED 6 YEARS AGO
11017
New Notebook
Download (69 MB)

Credit Card Fraud Detection

Anonymized credit card transactions labeled as fraudulent or genuine



Data Card
Code (4565)
Discussion (103)

About Dataset

Context

It is important that credit card companies are able to recognize fraudulent credit card transactions so that customers are not charged for items that they did not purchase.

Content

The dataset contains transactions made by credit cards in September 2013 by European cardholders.

This dataset presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions.

Usability 8.53

License
Database: Open Database, Cont...

Expected update frequency
Not specified

Tags

Finance Crime

Este dataset lo cargaremos dentro de un python dataframe. Para ello, utilizaremos un fichero que ya está cargado anteriormente en un Google Drive público.

Para ello, utilizaremos los paquetes de python necesarios para conectarnos a Google Drive y descargar la información del fichero.

```
from pydrive2.auth import GoogleAuth
from pydrive2.drive import GoogleDrive
from google.colab import auth
from oauth2client.client import GoogleCredentials

auth.authenticate_user()

gauth = GoogleAuth()

gauth.credentials = GoogleCredentials.get_application_default()

drive = GoogleDrive(gauth)

downloaded = drive.CreateFile({'id':"1jN9Oj-KJt_3fu7mhEcGjbg-XVCWk5e-_"})

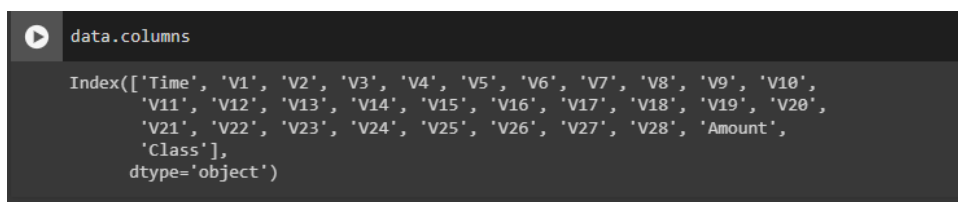
downloaded.GetContentFile('creditcard.csv')
```

Una vez descargado el fichero, lo cargamos en memoria a través de un dataframe. Para ello, primero cargamos el paquete pandas y posteriormente lo usamos para la lectura del fichero.

```
import pandas as pd

data = pd.read_csv('creditcard.csv')
```

En este fichero tenemos 28 variables que describen cada transacción de las tarjetas de crédito, el tiempo, la cantidad de dinero y si ha sido detectado como fraude o no.

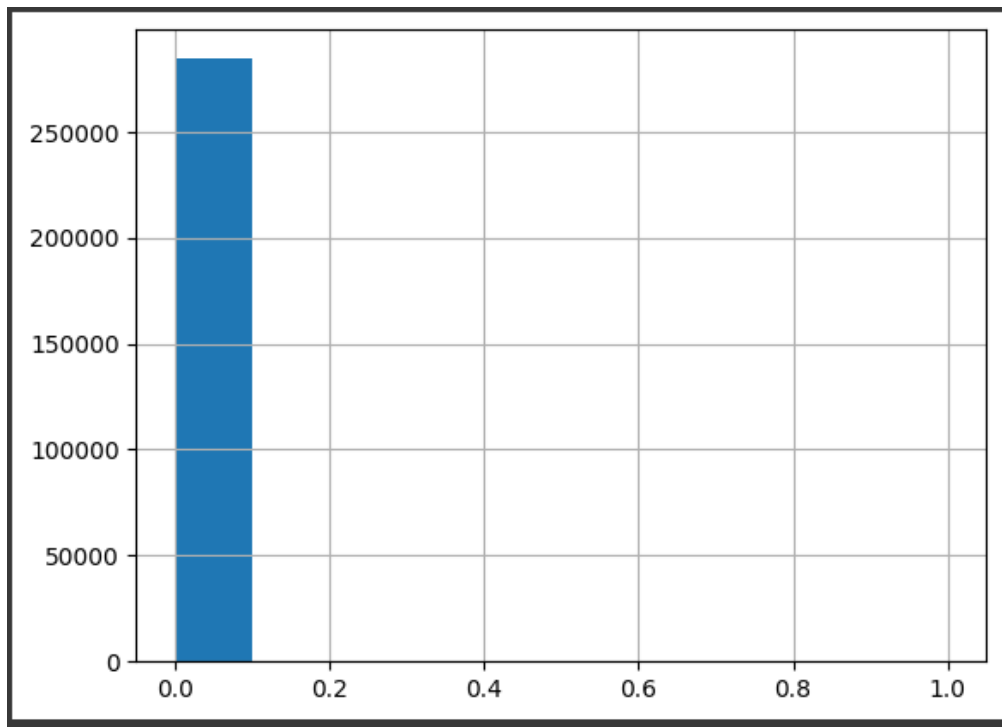


```
data.columns

Index(['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10',
      'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20',
      'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Amount',
      'Class'],
      dtype='object')
```

El objetivo es realizar un modelo de predicción de detección de fraude en tarjetas de crédito. Para conocer cuáles son fraude con basamos en la columna "Class". Las filas que tengan un uno en esa columna son transacciones consideradas como fraude.

Como se puede apreciar la muestra está muy balanceada.



Donde tenemos

```
data.Class.value_counts()
0    284315
1         492
Name: Class, dtype: int64
```

6.2. Modelo básico

Vamos a comparar si estas técnicas sirven para algo. Para ello, vamos primero a hacer un modelo básico con la información de la que disponemos.

Utilizaremos un modelo de Random Forest para poder predecir el movimiento de la tarjeta es fraude o no.

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

sklearn.ensemble.RandomForestClassifier

```
class sklearn.ensemble.RandomForestClassifier(n_estimators=100, *, criterion='gini', max_depth=None,
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='sqrt', max_leaf_nodes=None,
min_impurity_decrease=0.0, bootstrap=True, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False,
class_weight=None, ccp_alpha=0.0, max_samples=None, monotonic_cst=None) [source]
```

A random forest classifier.

A random forest is a meta estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting. Trees in the forest use the best split strategy, i.e. equivalent to passing `splitter="best"` to the underlying `DecisionTreeRegressor`. The sub-sample size is controlled with the `max_samples` parameter if `bootstrap=True` (default), otherwise the whole dataset is used to build each tree.

For a comparison between tree-based ensemble models see the example [Comparing Random Forests and Histogram Gradient Boosting models](#).

Read more in the [User Guide](#).

Para ello, primero dividimos la muestra en dos grupos de entrenamiento y en test:

```
Y = data['Class']
X = data.drop('Class', axis=1)
X_train, X_test, y_train, y_test = train_test_split(X, Y, train_size=0.7)
```

En paralelo, creamos un clasificador random Forest con el siguiente línea:

```
cls = LogisticRegression ()
```

Tras la creación del clasificador lo vamos a entrenar con la muestra que tenemos:

```
cls.fit(X_train, y_train)
```

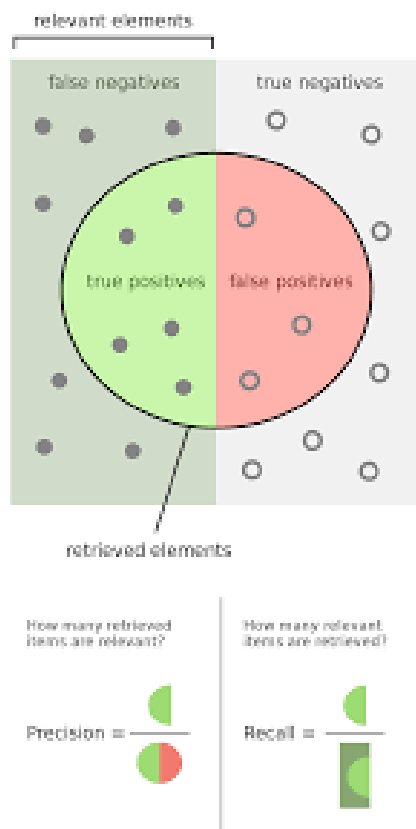
Una vez entrenado, intentamos utilizar la muestra de test para validar la calidad del modelo desarrollado.

```
y_pred = cls.predict(X_test)
```

Y ya teniendo las predicciones y los valores que deberíamos tener podemos, predecir el accuracy del modelo.

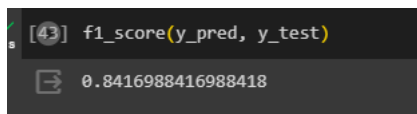
```
accuracy = accuracy_score(y_pred, y_test)
accuracy
0.9995201479348805
```

El accuracy o precisión es una medida muy peligrosa para los modelos muy balanceados.



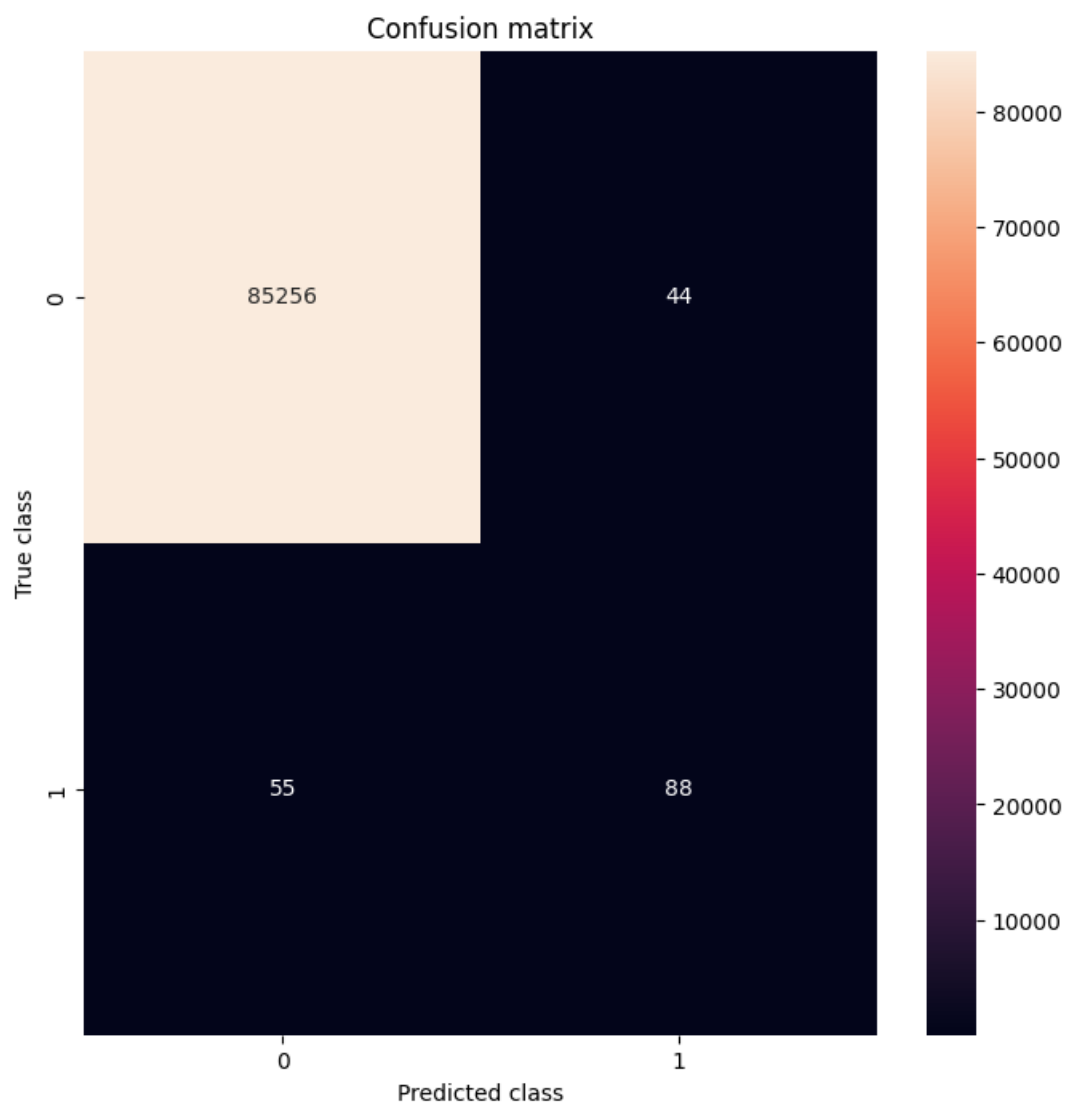
Como podemos ver en esta definición de la precisión tenemos una división entre los valores correctamente dictaminados y el total. Cuando la mayoría de los valores son 0, entonces la precisión es muy alta porque casi todos los valores son 0. Para el modelo es más fácil devolver todos como 0, que intentar dividir correctamente la muestra.

Por eso, utilizaremos muestras como el F1 y el recall.



```
[43]: f1_score(y_pred, y_test)
0.8416988416988418
```

Si vemos la matriz de confusión, vemos que el accuracy se debe a que hay muchos 0 y, por tanto, casi siempre acierta.



6.3. Modelo NearMiss

Para compararlo con ese modelo básico vamos a utilizar una técnica conocida como NearMiss para poder seleccionar las muestras de entrenamiento de forma correcta.

NearMiss

```
class imblearn.under_sampling.NearMiss(*, sampling_strategy='auto',
version=1, n_neighbors=3, n_neighbors_ver3=3, n_jobs=None) [source]
```

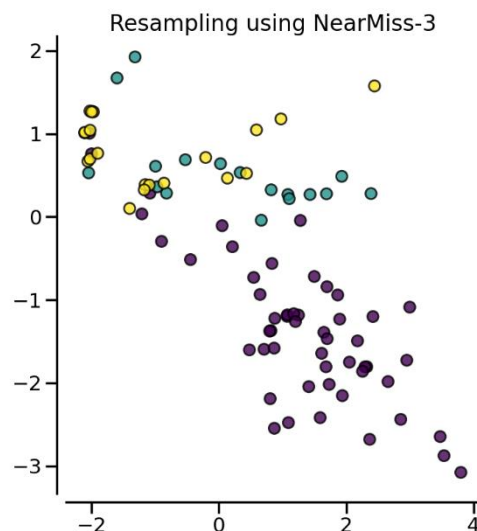
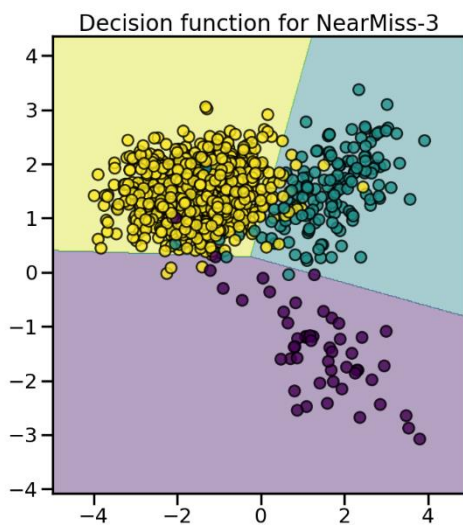
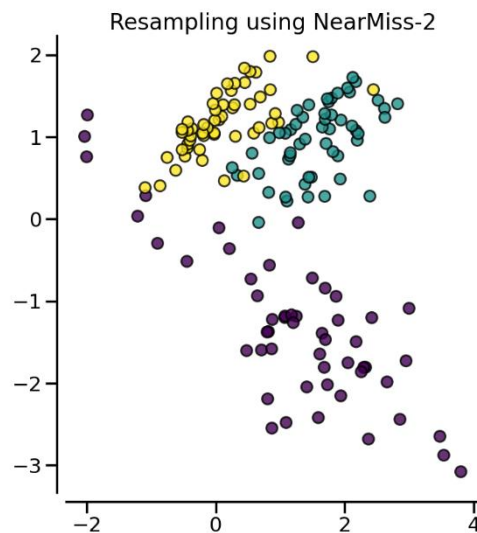
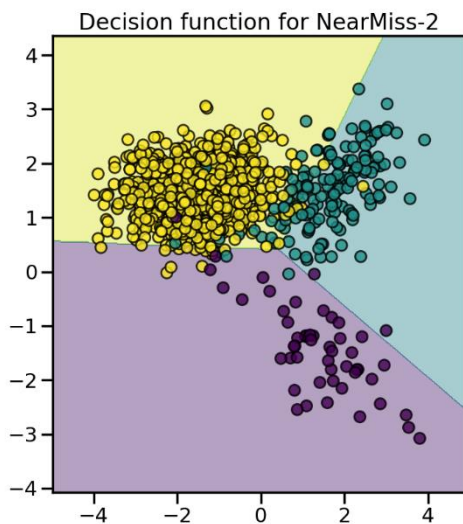
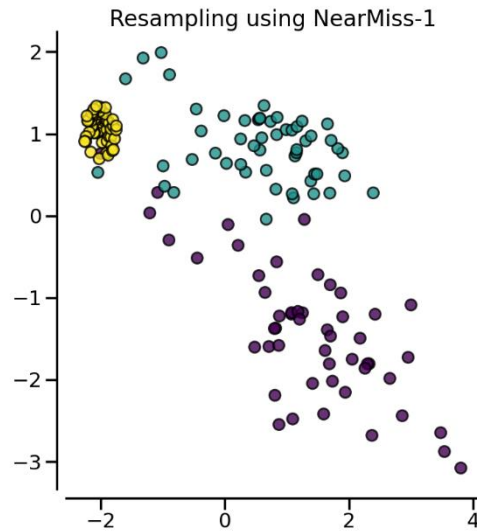
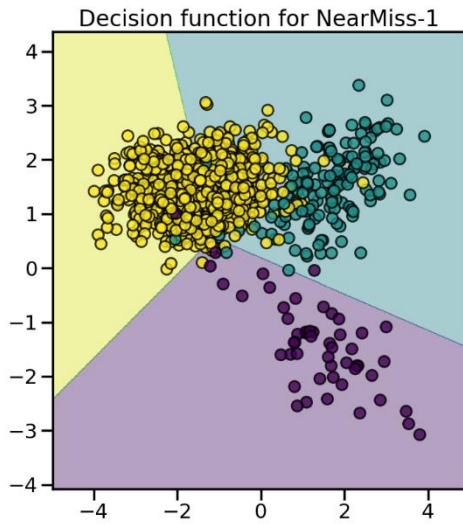
Class to perform under-sampling based on NearMiss methods.

Read more in the [User Guide](#).

Parameters:

Esta técnica, lo único que hacer es reducir el dataset de entrenamiento, seleccionando de la muestra mayoritaria las muestras más cercanas a las de la muestra minoritaria.

Hay tres técnicas de realización, que están explicadas en la siguiente página web: https://imbalanced-learn.org/stable/auto_examples/under-sampling/plot_comparison_under_sampling.html#sphx-glr-auto-examples-under-sampling-plot-comparison-under-sampling-py



Solo haremos una prueba de concepto con una de las técnicas.

Para ello, utilizaremos un dataset tuneado con el NearMiss.

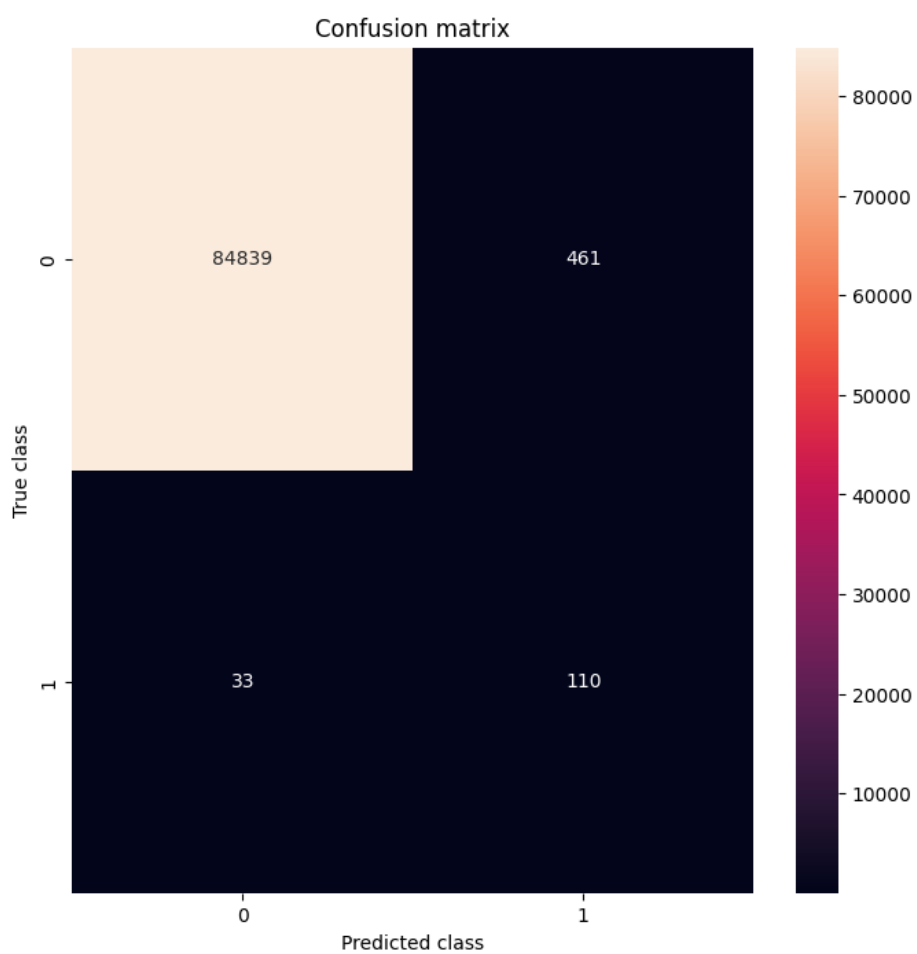
```
us = NearMiss(sampling_strategy=0.01, n_neighbors=3, version=2)  
X_train_res, y_train_res = us.fit_resample(X,Y)
```



Con esta línea hemos balanceado la nueva muestra a un 10 a 1. Es decir, tenemos 10 muestras con 0 por cada una con 1.

```
{0: 4920, 1: 492}
```

Mirando la nueva matriz de confusión, vemos que mejoramos el número de aciertos, frente al modelo básico.



7. Sobre muestreo

Con el mismo dataset vamos a hacer un sobre muestreo la muestra para poder compararlo con el caso anterior en el que estábamos reduciendo el número de ceros. En este caso vamos a aumentar el número de unos de la clase class para poder ver cómo funciona el over Sampling.

Para ello, vamos a utilizar como en el caso anterior la librería de <https://imbalanced-learn.org/> para que podamos crear las nuevas muestras.

Utilizaremos la función:

RandomOverSampler

```
class imblearn.over_sampling.RandomOverSampler(*,
sampling_strategy='auto', random_state=None, shrinkage=None) [source]
```

Class to perform random over-sampling.

Object to over-sample the minority class(es) by picking samples at random with replacement. The bootstrap can be generated in a smoothed manner.

Read more in the [User Guide](#).

Igual que en el caso anterior, lo que vamos a hacer es modificar la muestra de entrenamiento y, por lo tanto, modificar el modelo para que detecte mejor las muestras que contienen fraude respecto al resto.

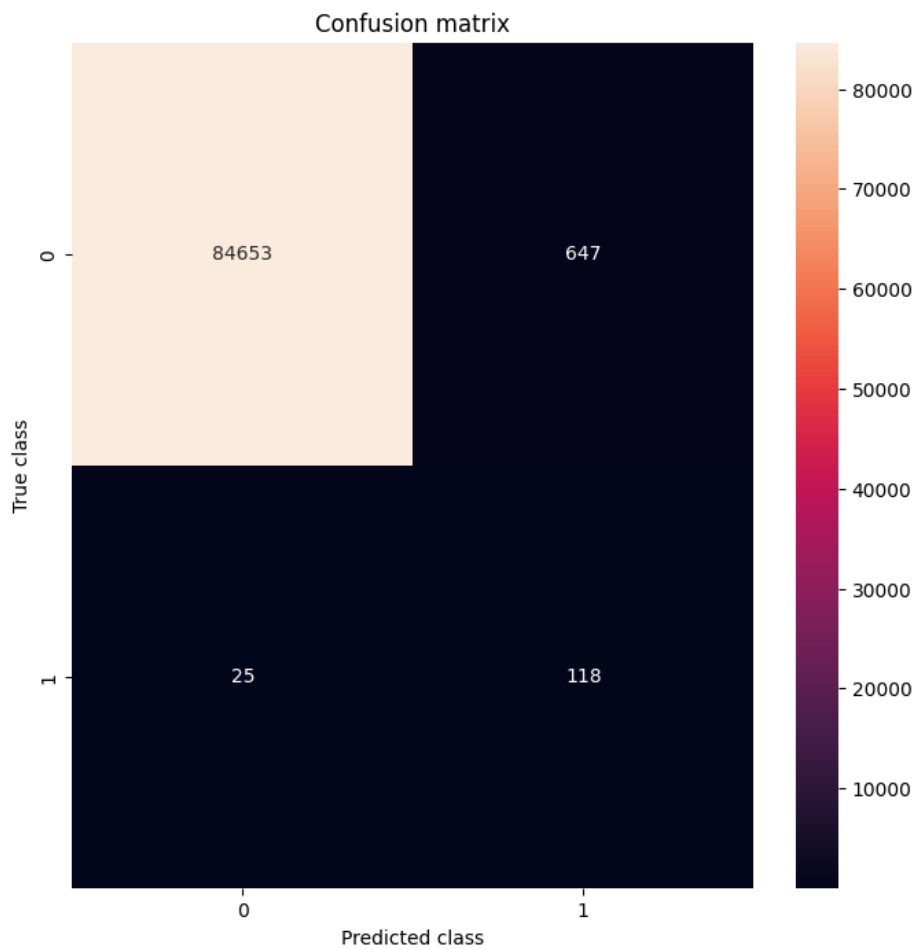
```
os = RandomOverSampler(sampling_strategy=0.1)
X_train_res, y_train_res = os.fit_resample(X_train, y_train)
```

Con este función lo que creo es una muestra para el entrenamiento con un ratio de muestreo de 10 a 1, igual que en el caso anterior.

```
0: 199015, 1: 19901
```

Como podemos ver en la comparación de la columna "Class" es de 10 a 1, tal y como hemos predicho. En este caso, lo que hacemos es crecer en la muestra a analizar.

Hecho lo mismo, la matriz de confusión resulta la siguiente:



8. Notebook

```
# -*- coding: utf-8 -*-
```

```
"""Subsampling.ipynb
```

Automatically generated by Colaboratory.

Original file is located at

https://colab.research.google.com/drive/1c0bqA9PgP9wkEVLV4ZOoItvvAhPPOn_9

```
"""
```

```
from pydrive2.auth import GoogleAuth
```

```
from pydrive2.drive import GoogleDrive
```

```
from google.colab import auth
```

```
from oauth2client.client import GoogleCredentials
```

```
auth.authenticate_user()
```

```
gauth = GoogleAuth()
```

```
gauth.credentials = GoogleCredentials.get_application_default()
```

```
drive = GoogleDrive(gauth)
```

```
downloaded = drive.CreateFile({'id': '1jN9Oj-KJt_3fu7mhEcGjbg-XVCWk5e-_'})
```

```
downloaded.GetContentFile('creditcard.csv')
```

```
import pandas as pd
```

```
data = pd.read_csv('creditcard.csv')
```

```
data.columns
```

```
data['Class'].hist()
```

```
data.Class.value_counts()
```

```
"""#modelo básico"""
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix
```

```
from sklearn.linear_model import LogisticRegression
```

```
from imblearn.under_sampling import NearMiss
```

```
from sklearn.model_selection import train_test_split
```

```
from collections import Counter
```

```
Y = data['Class']
```

```
X = data.drop('Class', axis=1)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, Y, train_size=0.7)
```

```
cls = LogisticRegression()
```

```
cls.fit(X_train, y_train)
```

```
y_pred = cls.predict(X_test)
```

```
accuracy = accuracy_score(y_pred, y_test)
```

```
accuracy
```

```
f1_score(y_pred, y_test)
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
conf_matrix = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(8, 8))
```

```
sns.heatmap(conf_matrix, annot=True, fmt="d");
```

```
plt.title("Confusion matrix")
```

```
plt.ylabel('True class')
```

```
plt.xlabel('Predicted class')
```

```
plt.show()
```

```
""" #NearMiss"""
```

```
us = NearMiss(sampling_strategy=0.01, n_neighbors=3, version=2)
```

```
X_train_res, y_train_res = us.fit_resample(X,Y)
```

```
print('Resampled dataset shape %s' % Counter(y_train_res))
```

```
cls1 = LogisticRegression()

cls1.fit(X_train_res, y_train_res)

y_pred = cls1.predict(X_test)

accuracy = accuracy_score(y_pred, y_test)
accuracy

f1_score(y_pred, y_test)

import matplotlib.pyplot as plt
import seaborn as sns

conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 8))
sns.heatmap(conf_matrix, annot=True, fmt="d");
plt.title("Confusion matrix")
plt.ylabel('True class')
plt.xlabel('Predicted class')
plt.show()

"""#oversampling"""

from imblearn.over_sampling import RandomOverSampler
```



```
os = RandomOverSampler(sampling_strategy=0.1)
X_train_res, y_train_res = os.fit_resample(X_train, y_train)

print('Resampled dataset shape %s' % Counter(y_train_res))

cls1 = LogisticRegression()

cls1.fit(X_train_res, y_train_res)

y_pred = cls1.predict(X_test)

accuracy = accuracy_score(y_pred, y_test)
accuracy

f1_score(y_pred, y_test)

import matplotlib.pyplot as plt
import seaborn as sns
```

```
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 8))
sns.heatmap(conf_matrix, annot=True, fmt="d");
plt.title("Confusion matrix")
plt.ylabel('True class')
plt.xlabel('Predicted class')
plt.show()
```

The background is a blue-tinted photograph of a university campus. In the center, there is a large, modern building with many windows. In the foreground, several students are walking on a paved path. A signpost with a lamp is visible on the left, and a banner hanging from it says "WELCOME TO UAX".

UAX

Universidad
Alfonso X el Sabio

G R A C I A S